SAHAYAK After-Training, Testing & New Sign Deployment Guide

Release checklist for validating a new static or continuous sign, promoting the correct candidate, syncing only approved model files, and verifying the deployed API.

Scope	Safe-by-default sequence
Windows local training project plus GitHub deployment repository.	Backup → collect → train candidate → test → promote → sync → Git commit/push → verify the deployed API.

Purpose and stop/go rule

Use this guide after collecting and training a new static or continuous sign. It is a release checklist: a candidate must pass tests and local recognition checks before promotion, synchronization, or GitHub push. The static and continuous lanes are separate; do not mix their model files or datasets.

Do not promote or push a candidate just because training completed. Stop if any previous label disappears, tests fail, the API does not load the model, or old signs regress.

1. Open the training project and confirm the scripts

```
Set-Location "C:\Users\user\3D Objects\New folder\SAHAYAK"
.\.venv\Scripts\Activate.ps1
python --version
Test-Path .\collect_data.py
Test-Path .\collect_sequences.py
Test-Path .\train_model.py
Test-Path .\train_sequence_model.py
```

If activation is blocked, keep the existing environment unchanged and replace python with .\.venv\Scripts\python.exe in the commands.

2. Back up both lanes before changes

```
$stamp = Get-Date -Format "yyyyMMdd_HHmmss"
$backup = Join-Path $PWD "backups\before_release_$stamp"
New-Item -ItemType Directory -Path $backup -Force | Out-Null
foreach ($file in @(
  "data\landmarks_2hand.csv",
  "continuous_gesture_db.db",
  "model\sahayak_twohand.joblib",
  "model\labels_twohand.json",
  "model\sahayak_sequence.joblib"
)) {
  if (Test-Path $file) { Copy-Item $file $backup -Force }
}
Write-Host "Release backup saved to: $backup"
```

Close any sequence collection process before backing up continuous_gesture_db.db. Do not delete, rename, overwrite, or use the legacy Code\gesture_db.db as the continuous dataset. This workflow uses continuous_gesture_db.db.

3A. Static sign lane: collect, train, and validate candidate

```
python collect_data.py
python train_model.py
python -m unittest test_sahayak.py
```

The static collector asks for one label and uses S to toggle saving, Q or Esc to exit. Training writes model\sahayak_twohand_candidate.joblib and model\labels_twohand_candidate.json. It does not replace the active model unless --promote is specified.

```
@'
import joblib, json
from pathlib import Path
new = joblib.load('model/sahayak_twohand_candidate.joblib')
labels = set(map(str, new.classes_))
old_path = Path('model/sahayak_twohand.joblib')
if old_path.exists():
    old = joblib.load(old_path)
    missing = set(map(str, old.classes_)) - labels
    assert not missing, f'Existing labels missing: {sorted(missing)}'
meta = set(json.loads(Path('model/labels_twohand_candidate.json').read_text(encoding='utf-8')))
assert getattr(new, 'n_features_in_', None) == 126
assert meta == labels
print('STATIC CANDIDATE VERIFIED:', sorted(labels))
'@ | python -
```

3B. Continuous sign lane: collect, train, and validate candidate

```
python collect_sequences.py
python -c "from sequence_db import summary; print(summary())"
python train_sequence_model.py --db
python -m unittest test_sequence.py test_sahayak.py
```

Continuous collection uses C to start one full movement and C again to stop/save. Training reads continuous_gesture_db.db and writes model\sahayak_sequence_candidate.joblib. It does not replace the active continuous model unless --promote is specified.

```
@'
import joblib
from pathlib import Path
new = joblib.load('model/sahayak_sequence_candidate.joblib')
labels = set(map(str, new.classes_))
old_path = Path('model/sahayak_sequence.joblib')
if old_path.exists():
    old = joblib.load(old_path)
    missing = set(map(str, old.classes_)) - labels
    assert not missing, f'Existing sequence labels missing: {sorted(missing)}'
assert getattr(new, 'n_features_in_', None) == 1320
assert len(labels) >= 2
print('CONTINUOUS CANDIDATE VERIFIED:', sorted(labels))
'@ | python -
```

4. Promote only the lane you have approved

```
# Static ONLY - run after inspecting the candidate report/tests:
python train_model.py --promote

# Continuous ONLY - run instead when promoting continuous signs:
python train_sequence_model.py --db --promote
```

Run one promotion command for the lane being released, not both automatically. Each training script backs up its active model before replacing it. Then start the local API:

```
python web_app.py
```

In a second PowerShell window, check both models and their labels:

```
$status = Invoke-RestMethod "http://127.0.0.1:5000/api/status"
$status | ConvertTo-Json -Depth 8
# Static readiness and labels:
$status.model_ready
$status.single_sign_labels
# Continuous readiness and labels:
$status.sequence_model_ready
$status.sequence_sign_labels
```

- Open http://127.0.0.1:5000 and test the newly added sign and representative previous signs. For a continuous sign, perform the whole movement from start to finish.
- Check confidence/uncertain behavior and moving hand annotations. A passing unit test is not enough; verify using real camera examples.
- If the relevant model is not ready or old labels fail, stop. Restore the previous model from the timestamped backup and do not sync or push.

5. Back up GitHub main and sync only the approved model

```
$TrainingRoot = "C:\Users\user\3D Objects\New folder\SAHAYAK"
$RepoPath = "C:\Users\user\3D Objects\New folder\SAHAYAK-ISL"

if (-not (Test-Path (Join-Path $RepoPath ".git"))) {
  git clone https://github.com/Mrjohanjyotikuli/SAHAYAK-ISL.git $RepoPath
}
git -C $RepoPath status --short
# Stop and resolve local changes before proceeding.
git -C $RepoPath pull --ff-only origin main
$backupBranch = "backup/pre-release-" + (Get-Date -Format "yyyyMMdd-HHmmss")
git -C $RepoPath branch $backupBranch main
git -C $RepoPath push origin $backupBranch
Set-Location $TrainingRoot
```

For a static update, stage only the active static model and its labels:

```
Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Static
git -C $RepoPath status --short
git -C $RepoPath add `
  SAHAYAK-Vercel-Deployment/backend/model/sahayak_twohand.joblib `
  SAHAYAK-Vercel-Deployment/backend/labels_twohand.json
git -C $RepoPath diff --cached --name-only
git -C $RepoPath commit -m "Update static sign recognition model"
git -C $RepoPath push origin main
```

For a continuous update, stage only the active sequence model:

```
Set-Location $TrainingRoot
.\sync_model_to_github.ps1 -RepoPath $RepoPath -ModelType Continuous
git -C $RepoPath status --short
git -C $RepoPath add SAHAYAK-Vercel-Deployment/backend/model/sahayak_sequence.joblib
git -C $RepoPath diff --cached --name-only
git -C $RepoPath commit -m "Update continuous sign recognition model"
git -C $RepoPath push origin main
```

Before commit, read git diff --cached --name-only. It must list only the expected model path(s). Never push data/landmarks_2hand.csv, continuous_gesture_db.db, gesture_db.db, .venv, candidate models, local backups, or credentials.

6. Verify production after Render finishes deploying

```
$status = Invoke-RestMethod "https://sahayak-backend-krh1.onrender.com/api/status"
$status.model_ready
$status.single_sign_labels
$status.sequence_model_ready
$status.sequence_sign_labels
```

- Confirm the correct readiness field is True and the new label appears in the matching label list.
- Open https://sahayak-eight-lac.vercel.app/studio.html and test the new sign plus existing signs in both modes.
- A model-only change is normally a backend deployment. If frontend source also changed, commit those frontend changes separately and verify the Vercel deployment.

Rollback

Never force-push main to undo a regression. Restore the approved backup model in the local training folder, sync that active model through the helper, stage only its expected deployment path, and push a clearly labelled rollback commit. Keep the timestamped backup branch to compare repository state.